

Análisis de Algoritmos

Ciencias de la Computación

Abril – Agosto / 2025



Unidad 3

2. Análisis de algoritmos

2.1 Estructuras de control

2.2 Análisis del caso medio

2.3. Análisis amortizado

2.4. Recurrencias



Semana 6

Estructuras de control

- ¿Qué son las estructuras de control?
 - Conjunto de instrucciones que permiten modificar el flujo de ejecución de un algoritmo.
 - Permiten que el algoritmo tome decisiones, repita procesos o siga una secuencia ordenada.
 - Se clasifican en: Secuenciales, Condicionales y Repetitivas.

Estructuras de control

- Estructuras secuenciales
 - Instrucciones que se ejecutan una tras otra en el orden en que se escriben.
 - No requieren decisiones ni condiciones.
 - Base para estructuras más complejas.

```
Leer A
Leer B
C ← A + B
Imprimir C
```

Estructuras de control

- Estructura de control iterativa: Bucles (para)
 - Repite un bloque de instrucciones un número determinado de veces.
 - Se utiliza cuando se conoce de antemano cuántas veces debe repetirse el ciclo.

```
Para i ← 1 hasta n hacer  
    Instrucción(es)  
FinPara
```

Estructuras de control

- **Llamadas recursivas – ¿Qué es la recursión?**
 - Técnica en la cual una función se llama a sí misma para resolver un problema.
 - Útil cuando el problema puede dividirse en subproblemas similares más pequeños.
 - Elementos:
 - Caso base: Condición para detener la recursión.
 - Caso recursivo: La función se llama a sí misma con una entrada modificada.

Estructuras de control

- Llamadas recursivas
 - Ejemplo:

```
Función factorial(n)
    Si n = 0 entonces
        retornar 1
    Sino
        retornar n * factorial(n - 1)
FinFunción
```


Estructuras de control

- Bucle mientras – Definición
 - Estructura de control que repite un bloque de instrucciones mientras se cumpla una condición booleana.
 - Se usa cuando no se sabe cuántas veces debe repetirse el ciclo.

```
x ← 1
Mientras x ≤ 5 hacer
    Imprimir x
    x ← x + 1
FinMientras
```

Si $n=5$

Matemáticamente

$$\sum_{i=1}^n (\mathbf{1}) = n$$

$$\sum_{i=1}^n (1) = 1 + 1 + 1 + 1 + 1 = 5$$

Ejemplo

```
12   public static void CicloFor2(int n) {  
13      int contador = 0;  
14      for (int i=1; i<=n; i=i*2) {  
15          contador++;  
16      }  
17 }
```

contador **se incrementa sólo cuando:**

$i = 1, 2, 4, 8, 16, \dots, 2^k$ donde $k = \log_2 n$

$t(n) = O(\log n)$

n	contador
1	1
2	2
3	2
4	3
5	3
6	3
7	3
8	4
9	4
10	4
11	4
12	4

2^k

Ejemplo

```

procedure burbuja ( var A: array [1..n] of integer );
    { burbuja clasifica el arreglo A de menor a mayor }
var
    i, j, temp: integer;
begin
(1)     for i := 1 to n-1 do
(2)         for j := n downto i + 1 do
(3)             if A[j-1] > A[j] then begin
(4)                 { intercambia A[j-1] y A[j] }
(5)                 temp := A[j-1];
(6)                 A[j-1] := A[j];
(6)                 A[j] := temp
                end
    end; { burbuja }
  
```

}

$n - i$

$$\begin{aligned}
 \sum_{i=1}^{n-1} (n - i) &= \sum_{i=1}^n n - \sum_{i=1}^n i \\
 &= \frac{n^2}{2} - \frac{n}{2}
 \end{aligned}$$

n=5

i	j
1	5, 4, 3, 2
2	5, 4, 3
3	5, 4
4	5

10 iteraciones

Taller

- Calcular $T(n)$ de los siguientes algoritmos:
 - Codificar la aplicación en java
 - Obtener el $t(n)$ en base al análisis
 - Comparar el modelo obtenido con los resultados de la aplicación.

Taller

```
procedure misterio(n:integer)
var
    contador, i, j, k : integer

begin
    contador:=0
    for i:=1 to n-1 do
        for j:=i+1 to n do
            for k:=1 to j do
                contador:=contador + 1
            end
        end
    end
end
```

Taller – Algoritmo 1

```
procedure prod_mat ( n: integer );  
  var  
    i, j, k: integer;  
begin  
  for i := 1 to n do  
    for j := 1 to n do begin  
       $C[i, j] := 0;$   
      for k := 1 to n do  
         $C[i, j] := C[i, j] + A[i, k] * B[k, j]$   
      end  
    end  
  end  
end
```



Semana 7

Recurrencias

- El tiempo de ejecución para un **algoritmo recursivo** está más fácilmente expresado por una expresión recursiva.
- Una relación de recurrencia define una función mediante una expresión que incluye una o más instancias (más pequeñas) de si misma. Por ejemplo: $n!$
- Las ecuaciones de recurrencia son utilizadas para determinar cotas asintóticas en algoritmos que presentan recursividad.

Recurrencias

- Solución:
 - Suposiciones inteligentes
 - Ecuación característica
- Suposiciones inteligentes
 1. Calcular los primeros valores de la recurrencia
 2. Buscar una regularidad
 3. Inventar una forma general
 4. Demostrar por inducción matemática (u otro método).

Recurrencias

```
5 public static long Factorial(long n) {
6     if (n == 0)
7         return 1;
8     else
9         return n * Factorial(n-1);
10 }
```

$$t(n) = \begin{cases} 0 & n = 0 \\ t(n-1) + 1 & n > 0 \end{cases}$$

Recurrencias

n	T(n)	
1	$T(1) = T(1-1) + 1$	$T(0) + 1 = 1$
2	$T(2) = T(2-1) + 1$	$T(1) + 1 = 2$
3	$T(3) = T(3-1) + 1$	$T(2) + 1 = 3$
4	$T(4) = T(4-1) + 1$	$T(3) + 1 = 4$
5	$T(5) = T(5-1) + 1$	$T(4) + 1 = 5$

$$= T(n - 1) + 1$$

$$= (n-1)+1$$

$$= n$$

Recurrencias

$$T(n) = \begin{cases} 0 & \text{Si } n = 0 \\ 3T(n/2) + n & \text{Caso contrario} \end{cases}$$

	n	T(n)
$2^0 \longrightarrow$	1	1
$2^1 \longrightarrow$	2	$3T(2/2) + 2 = 3T(1) + 2 = 5$
$2^2 \longrightarrow$	4	$3T(4/2) + 4 = 3T(2) + 4 = 19$
$2^3 \longrightarrow$	8	$3T(8/2) + 8 = 3T(4) + 8 = 65$
$2^4 \longrightarrow$	16	$3T(16/2) + 16 = 3T(8) + 16 = 211$
$2^5 \longrightarrow$	32	$3T(32/2) + 32 = 3T(16) + 32 = 665$

Recurrencias

$$2 = 3 * 1 + 2$$

$$2^2 = 3(3 * 1 + 2) + 2^2 = 3^2 * 1 + 3 * 2 + 2^2$$

$$2^3 = 3(3^2 * 1 + 3 * 2 + 2^2) + 2^3 = 3^3 * 1 + 3^2 * 2 + 3 * 2^2 + 2^3$$

$$2^4 = 3(3^3 * 1 + 3^2 * 2 + 3 * 2^2 + 2^3) + 2^4$$
$$= 3^4 * 1 + 3^3 * 2 + 3^2 * 2^2 + 3 * 2^3 + 2^4$$

$$T(2^k) = 3^k * 2^0 + 3^{k-1} * 2^1 + 3^{k-2} * 2^2 \dots + 3^1 * 2^{k-1} + 3^0 * 2^k$$

$$\sum_{i=0}^k 3^{k-i} 2^i = 3^k \sum_{i=0}^k (2/3)^i = 3^{k+1} - 2^{k+1}$$

Taller

- Resolver la siguiente recurrencia

$$T(n) = \begin{cases} 0 & \text{si } n=0 \\ 2T(n-1) + 1 & \text{si } n > 0 \end{cases}$$

Taller 2

- Codificar el algoritmo de Fibonacci
- Identificar las recurrencias
- Obtener la ecuación general
- Demostrar